

Detailed Explanation of run_oc_simple_time_sim_arbitrary.m

Generated from the uploaded MATLAB files: run_oc_simple_time_sim_arbitrary.m, my_model(14).m, my_params(22).m, and README_SIMPLE_TIME_SIM_V3_CHECKED.md.

Purpose in one sentence: this file lets you time-simulate the vehicle model from my_model.m without running the optimal-control transcription, result.mat, collocation, or IPOPT. It compares three numerical paths: ode15s reduced ODE, fixed-step RK4 reduced ODE, and ode15i true DAE.

Important spelling note: your uploaded file is named run_oc_simple_time_sim_arbitrary.m. Your message wrote arbitray.m, but the actual file and function use arbitrary.

1. Executive summary

The simulator is organized around the distinction between differential states x and algebraic tyre vertical loads z . In my_model.m, x has 13 scaled states and z has 4 scaled tyre loads: Fz_FL, Fz_FR, Fz_RL, Fz_RR. The model exposes y_xdot for time-domain state derivatives and h_load for the four Chapter-9 tyre-load algebraic residuals. The simulator can either eliminate z with a Newton solve at every RHS call, or keep z inside a true DAE solved by ode15i.

The three numerical paths are:

Path	What it integrates	How tyre loads are handled	Main accuracy knobs
ode15s reduced ODE	Only $x(t)$	Your Newton solver solves $h_load=0$ inside each RHS call.	opts.ode_reltol, opts.ode_abstol, opts.ode_max_step, opts.output_dt for saved outputs.
Fixed RK4 reduced ODE	Only $x(t)$, on uniform grid	Your Newton solver solves $h_load=0$ four times per RK4 step.	opts.fixed_dt, opts.load_tol, opts.load_max_iter. Cost grows as $1/\text{fixed_dt}$.
ode15i true DAE	$y(t)=[x(t); z(t)]$	z is a solver variable. No call to solve_loads_newton inside the residual; MATLAB solves the coupled DAE internally.	opts.dae_reltol, opts.dae_abstol, opts.dae_max_step, decic diagnostics.

2. Relationship to my_model.m and my_params.m

The simulator calls my_params(), then runs my_model.m. In your uploaded my_model.m, the states are scaled, physical variables are recovered by multiplying by x_s , and the algebraic variables z are only the four tyre vertical loads. The state vector order used by the simulator is $u_x, u_y, r, \phi, \dot{\phi}, \theta, \dot{\theta}, n, x_i, \omega_{fl}, \omega_{fr}, \omega_{rl}, \omega_{rr}$. The control vector order is drive torque T_t , brake torque T_b , and steering δ . The algebraic vector order is $fz_{fl}, fz_{fr}, fz_{rl}, fz_{rr}$.

A key detail is that my_model.m computes both a time-domain $\dot{x}$ and a distance-domain $dx = sf \cdot \dot{x}$, but sets $y_xdot = \dot{x}$. Therefore this time simulator uses time-domain dynamics. This is correct for a time simulation and different from the distance-domain transcription used in an optimal-control lap problem.

The vehicle parameters in my_params.m define mass, inertia, geometry, suspension roll/pitch parameters, tyre model coefficients, scaling factors, bounds, torque and steering limits, and power limit. The simulator does not create new physics; it calls the model functions built from those symbolic expressions.

3. Whole-file execution flow

```
profile + user_opts
-> default input handling
-> build_model_functions_from_my_model()
   -> my_params()
   -> run('my_model.m')
```

```

-> CasADi functions: f_xdot, f_hload, dh_load/dz, diagnostics
-> default_options(user_opts, veh)
-> make_arbitrary_controls(profile, opts, veh)
-> make_initial_state_scaled()
-> static_load_guess_scaled()
-> solve h_load=0 once for reduced-ODE initial z
-> ode15s reduced ODE
-> fixed RK4 reduced ODE
-> optional ode15i true DAE
-> postprocess and plot

```

4. Detailed source-order walkthrough

Lines	Code region	Detailed meaning
1-19	File header and examples	Documents the purpose of the file: run a simple time simulation from my_params.m and my_model.m only. It explicitly avoids my_nlp.m, result.mat, collocation, and IPOPT. The examples show a default run and a shorter run with opts.t_end, opts.fixed_dt, and opts.do_plot.
21-26	Input defaults	If profile is missing, the simulator uses mild_sine. If user_opts is missing, it creates an empty struct so default_options can safely merge defaults and user overrides.
28-35	Setup	Prints run information, builds CasADi functions from my_model.m, extracts veh, constructs options, and creates arbitrary control histories. This is the bridge from the Chapter-9 model expressions to executable simulation functions.
37-46	Initial conditions	The state initial condition x0 is physical first and then scaled. The tyre-load initial guess z0 is static-load based. For ode15s and RK4, z0 is improved with one Newton solve of h_load=0. For ode15i, the static z0 is intentionally passed to decic so MATLAB finds a consistent algebraic initial condition.
48-65	ode15s reduced ODE path	ode15s integrates only x. At each RHS call, rhs_scaled solves the algebraic tyre-load equations using solve_loads_newton, then evaluates f_xdot. Therefore ode15s does not directly integrate z; z is a recomputed algebraic quantity. output_dt sets requested outputs, while ode_reltol, ode_abstol, and ode_max_step control variable-step accuracy.
67-73	Fixed RK4 path	simulate_fixed_rk4 advances x on the uniform grid fixed_dt. Each step has four RHS stages, and each stage solves h_load=0. Computational cost scales roughly as 4*t_end/fixed_dt Newton solves plus postprocessing solves.
75-86	ode15i true DAE path	If opts.run_dae is true, ode15i integrates y=[x;z] with the implicit residual [xdot - f_xdot; h_load]. The user Newton load solver is not called inside dae_residual_scaled; MATLAB internally solves the coupled nonlinear DAE system.
88-98	Diagnostics and plotting	The driver prints max absolute h_load residuals and optionally plots speed, selected states, commands, tyre loads, load residuals, and speed differences relative to the true DAE result.
102-137	build_model_functions_from_my_model	Runs my_params and my_model in a local workspace, checks required variables, stores scaling vectors and bounds, and creates single-output CasADi functions. The use of y_xdot is important: in your uploaded my_model.m, y_xdot is the time-domain derivative, whereas dx is the distance-domain derivative.
140-168	default_options	Defines simulator defaults. The most important knobs are t_end, output_dt, fixed_dt, ode tolerances, DAE tolerances, maximum solver steps, run_dae, enforce_state_bounds_fixed, kappa_constant, and Newton load-solve settings.
171-194	make_arbitrary_controls	Generates command histories on the output_dt grid. straight_accel has zero steering; mild_sine has small steering and torque ripple; slalom has larger and faster steering. The torque and steering commands are saturated to vehicle bounds. Brake command is stored as zero.
197-215	make_initial_state_scaled	Sets ux0=5 m/s, zero lateral/yaw/roll/pitch/path states, and wheel speeds equal to ux0/rw. Then it scales by x_s and clamps to state bounds.
218-224	static_load_guess_scaled	Computes static front and rear axle loads from m*g and the wheelbase distribution, splits each axle equally between left and right tyres, scales by z_s, and clamps to load bounds.
227-234	rhs_scaled	This is the reduced-ODE RHS used by both ode15s and RK4. The order is: evaluate curvature, apply controls, solve loads, evaluate xdot. That order matters because tyre forces and Chapter-9 load equations depend on z.
237-253	make_applied_control_scaled	Interpolates commands and enforces motor torque limits. It computes rear axle speed, converts motor power limit into torque limit, clips drive torque, hard-codes brake torque to zero, clips steering, and scales the final control vector.
256-264	eval_kappa	Returns curvature. By default it is constant kappa_constant=0. If opts.kappa_fun is supplied, the simulator calls it as kappa_fun(t,xs).

Lines	Code region	Detailed meaning
267-336	solve_loads_newton	Solves the four algebraic equations $h_load(x,u,z,kappa)=0$. It starts from the previous load vector, clamps to bounds, uses the analytic CasADi Jacobian dh_load/dz , applies damped Newton with backtracking, protects against singular Jacobians with pinv, and warns if the final residual is too large.
339-366	simulate_fixed_rk4	Implements classical RK4 explicitly. For each time step it computes k1, k2, k3, k4; each k uses rhs_scaled and therefore solves tyre loads. State clamping is off by default to keep the model comparable with ode15s and ode15i.
369-430	simulate_ode15i_dae	Builds the DAE vector $y=[x;z]$, asks decic to keep x0 fixed while adjusting z0 and xdot0, then runs ode15i. Diagnostics include decic_residual_inf and initial_z_shift_phys.
433-447	dae_residual_scaled	Defines exactly the DAE residual ode15i must drive to zero: $xdot - f_xdot$ for the 13 differential equations, and h_load for the 4 algebraic equations.
450-491	postprocess_solution	For reduced-ODE results, reconstructs z at every saved time using Newton, evaluates diagnostics, and stores scaled and physical outputs.
494-531	postprocess_dae_solution	For DAE results, uses the z returned by ode15i directly. It evaluates h_load only as a diagnostic rather than re-solving it.
534-617	plot_simple_results	Creates comparison figures. The plots are diagnostic only; they do not affect numerical integration.

5. How to run and play with the simulations

Use clear functions and rehash after editing local functions, because MATLAB can cache function definitions.

```
clear functions
rehash
addpath('D:\casadi_3_7_2') % adjust to your CasADi folder
addpath('D:\Applications\TOTPT-main\oc') % adjust to the folder containing my_model.m/my_params.m

opts = struct();
opts.t_end = 3;
opts.fixed_dt = 1e-3;
opts.do_plot = true;
[sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl] = run_oc_simple_time_sim_arbitrary('mild_sine', opts);

max(abs(sim_ode15s.h_load(:)))
max(abs(sim_fixed.h_load(:)))
max(abs(sim_ode15i_dae.h_load(:)))
sim_ode15i_dae.decic_residual_inf
```

Choose a command profile

The file supports straight_accel, mild_sine, and slalom. These are not optimal-control trajectories; they are artificial torque and steering histories intended to exercise the model.

```
opts = struct('t_end', 5, 'fixed_dt', 1e-3, 'do_plot', true);
run_oc_simple_time_sim_arbitrary('straight_accel', opts);
run_oc_simple_time_sim_arbitrary('mild_sine', opts);
run_oc_simple_time_sim_arbitrary('slalom', opts);
```

To make your own profile, add another case inside make_arbitrary_controls. Use ctrl.Tt_cmd for drive torque in Nm and ctrl.delta_cmd for steering in rad. At present, brake torque is hard-coded to zero in make_applied_control_scaled, so changing ctrl.Tb_cmd alone will not activate braking. To play with braking, line 248 must be changed from Tb = 0.0 to an interpolated and saturated brake command.

Adjust simulation duration and saved output spacing

opts.t_end controls how many physical seconds are simulated. opts.output_dt controls saved output times for ode15s and ode15i, and also the grid on which the arbitrary control histories are generated. For RK4, output_dt does not set the internal step; fixed_dt does.

```
% Faster short debugging run
opts.t_end = 1.0;
opts.output_dt = 0.05;
opts.fixed_dt = 2e-3;
opts.run_dae = false;
opts.do_plot = false;

% Longer plotted comparison run
opts.t_end = 15.0;
opts.output_dt = 0.02;
opts.fixed_dt = 1e-3;
opts.run_dae = true;
opts.do_plot = true;
```

Tune fixed-step RK4

For RK4, `opts.fixed_dt` is the actual time step. Smaller `fixed_dt` usually gives better accuracy but costs more. Because each RK4 step uses four RHS stages, and each RHS stage solves the tyre-load algebraic equations, the nominal RHS count is $4 \cdot (t_{\text{end}}/\text{fixed_dt})$. For example, `t_end=3` and `fixed_dt=1e-3` gives about 12,000 RHS calls, before postprocessing.

```
% Medium RK4 accuracy
opts.fixed_dt = 1e-3;
```

```
% Higher RK4 accuracy, roughly 5x more steps than 1e-3
opts.fixed_dt = 2e-4;
```

```
% Faster but less accurate, useful only for rough checks
opts.fixed_dt = 5e-3;
```

A practical RK4 convergence check is to run `fixed_dt`, then `fixed_dt/2`, and compare selected physical outputs such as `ux`, `r`, `xi`, and `Fz`. For a smooth non-stiff problem with a well-converged load solve, RK4 error often shrinks strongly as `dt` is halved. In this vehicle problem the tyre saturation, load bounds, and stiffness can reduce ideal fourth-order behavior, so trust direct comparisons rather than assuming perfect order.

Tune ode15s reduced ODE

`ode15s` chooses internal steps automatically. The `tspan` vector only requests output times. Accuracy and runtime are mainly controlled by `opts.ode_reltol`, `opts.ode_abstol`, and `opts.ode_max_step`. If controls or curvature vary quickly, reduce `ode_max_step` even if tolerances are already tight.

```
% Default-ish
opts.ode_reltol = 1e-6;
opts.ode_abstol = 1e-8;
opts.ode_max_step = 0.02;
```

```
% Higher accuracy reference
opts.ode_reltol = 1e-8;
opts.ode_abstol = 1e-10;
opts.ode_max_step = 0.005;
```

```
% Faster rough check
opts.ode_reltol = 1e-4;
opts.ode_abstol = 1e-6;
opts.ode_max_step = 0.05;
```

Tune ode15i true DAE

`ode15i` solves the DAE residual directly. It is the cleanest mathematical representation of the lifted-load DAE, but it may be slower because MATLAB must solve a coupled nonlinear implicit problem. Use `sim_ode15i_dae.deci_residual_inf` to check whether the initial condition was made consistent. Use `sim_ode15i_dae.initial_z_shift_phys` to see how much the initial loads moved from the static guess.

```
opts.run_dae = true;
opts.dae_reltol = 1e-7;
opts.dae_abstol = 1e-9;
opts.dae_max_step = 0.01;
```

```
[sim_ode15s, sim_fixed, sim_dae] = run_oc_simple_time_sim_arbitrary('mild_sine', opts);
sim_dae.deci_residual_inf
sim_dae.initial_z_shift_phys
```

Tune the tyre-load Newton solve

The reduced ODE paths depend on `solve_loads_newton`. The main knobs are `opts.load_tol`, `opts.load_max_iter`, and `opts.warn_load_residual`. Tightening `load_tol` improves algebraic consistency but can slow the simulation. If `max(abs(sim.h_load(:)))` is not near the tolerance, increase `load_max_iter`, improve the initial guess, or reduce step sizes/aggressiveness.

```

opts.load_tol = 1e-10;
opts.load_max_iter = 20;
opts.warn_load_residual = 1e-7;

```

```

max(abs(sim_ode15s.h_load(:)))
max(abs(sim_fixed.h_load(:)))

```

Note: `opts.load_fd_rel` is present in `default_options`, but this checked version uses the analytic CasADi Jacobian `model.f_hload_jac_z`. Therefore `load_fd_rel` is currently a leftover option, not an active tuning knob.

Use DAE as a reference comparison

When `opts.run_dae=true`, the plotting routine already creates a speed-difference figure comparing `ode15s` and `RK4` against the DAE solution. You can make your own comparisons for any state or load by interpolating onto common times.

```

tx = sim_ode15i_dae.t;
ux_ode = interp1(sim_ode15s.t, sim_ode15s.x_phys(1,:), tx, 'linear', 'extrap');
ux_rk4 = interp1(sim_fixed.t, sim_fixed.x_phys(1,:), tx, 'linear', 'extrap');
ux_dae = sim_ode15i_dae.x_phys(1,:);
max(abs(ux_ode - ux_dae))
max(abs(ux_rk4 - ux_dae))

```

Change road curvature

By default the simulator is a straight-road time simulation with $\kappa=0$. You can set a constant curvature or provide a function of time and scaled state.

```

% Constant left-turn curvature, radius about 500 m
opts.kappa_constant = 1/500;

```

```

% Time-varying curvature function
opts.kappa_fun = @(t,xs) 0.002*sin(2*pi*0.1*t);

```

Because the arbitrary controls are not feedback controls, aggressive curvature or steering may drive the vehicle outside reasonable state ranges. That is useful for stress-testing, but do not interpret it as an optimal trajectory.

State-bound clamping

`opts.enforce_state_bounds_fixed` applies only to the fixed `RK4` path. It is false by default so all three integrators solve the same simulation model. Turning it on can prevent `RK4` from running into extreme values, but it changes the model relative to `ode15s` and `ode15i` because those solvers are not clamped during integration.

```

opts.enforce_state_bounds_fixed = true; % numerical guard only; not a physically identical comparison

```

6. Output structures and what to inspect

Field	Meaning
sim.t	Saved time samples in seconds.
sim.x_scaled	13 scaled states, one column per saved time.
sim.x_phys	Physical states. Rows: ux, uy, r, phi, phidot, theta, thetadot, n, xi, omega_fl, omega_fr, omega_rl, omega_rr.
sim.z_scaled	4 scaled tyre loads.
sim.z_phys	Physical tyre loads in Newtons: FL, FR, RL, RR.
sim.u_scaled	3 scaled applied controls.
sim.u_phys	Physical controls: Tt, Tb, delta.
sim.h_load	Four scaled load residuals. This is the most direct algebraic consistency diagnostic.
sim.acc	Acceleration diagnostic outputs from my_model.m: ax, ay, ux_dot, uy_dot, r_dot, phi_ddot, theta_ddot.
sim.torque	Wheel torque diagnostic outputs.
sim.mu	Tyre utilization ratio outputs.
sim.load	Load diagnostics: h_load, Chapter-9 predicted loads, longitudinal transfer DeltaZ, lateral transfer terms DeltaZ1 and DeltaZ2.

7. Practical accuracy workflow

For day-to-day debugging, start with a short run, no DAE, moderate RK4 step, and plotting off. For final checks, turn on DAE, tighten tolerances, and compare results. A useful workflow is:

Use case	Settings / checks
Fast smoke test	<code>opts.t_end=1; opts.run_dae=false; opts.fixed_dt=2e-3; opts.do_plot=false;</code>
Normal comparison	<code>opts.t_end=3; opts.run_dae=true; opts.fixed_dt=1e-3; opts.do_plot=true;</code>
High-confidence check	<code>opts.fixed_dt=2e-4; opts.ode_reltol=1e-8; opts.ode_abstol=1e-10; opts.dae_reltol=1e-8; opts.dae_abstol=1e-10;</code> <code>opts.ode_max_step=0.005; opts.dae_max_step=0.005;</code>
Main diagnostics	<code>max(abs(sim.h_load(:))), DAE decic_residual_inf, overlay plots, and halving fixed_dt to check convergence.</code>

8. Important caveats I would keep in mind

- Line 53 defines odefun but the code passes rhs_cached to ode15s. This is harmless but can be removed to avoid confusion.
- The z_cache in ode15s is an initial guess cache, not an integrated state. MATLAB may call the RHS at internal or rejected trial steps, so the cache is only a warm start for Newton, not a source of physical memory.
- ctrl.Tb_cmd is stored but the applied brake torque is hard-coded to zero. To test braking, change make_applied_control_scaled.
- output_dt is not the internal variable-step solver step. For ode15s and ode15i, MaxStep and tolerances govern internal steps. For RK4, fixed_dt is the internal step.
- The true DAE path avoids your explicit tyre-load Newton solve, but MATLAB still solves nonlinear equations internally as part of ode15i.
- Turning on enforce_state_bounds_fixed makes RK4 no longer exactly comparable with ode15s and ode15i, because only RK4 is clamped.

- The arbitrary profiles are open-loop. They are useful for model checks, but they are not a controller and not a lap-time solution.

Appendix A. Complete annotated source listing

This appendix lists every source line in `run_oc_simple_time_sim_arbitrary.m` with a line-specific explanation. The earlier sections give the deeper conceptual interpretation; this appendix is meant for line-by-line reading while you keep the MATLAB file open.

Main driver: `run_oc_simple_time_sim_arbitrary` (lines 1-99)

Line	Source	Explanation
1	<code>function [sim_ode15s, sim_fixed, sim_ode15i_dae, model, ctrl] = run_oc_simple_time_sim_arbitrary(profile, user_opts)</code>	This is the public entry point. It returns three simulation structures, the CasADi model wrapper, and the control command history.
2	<code>%RUN_OC_SIMPLE_TIME_SIM_ARBITRARY Time simulation using your my_model.m only.</code>	Comment/documentation: <code>RUN_OC_SIMPLE_TIME_SIM_ARBITRARY</code> Time simulation using your <code>my_model.m</code> only.
3	<code>%</code>	Comment spacer inside the file header or section.
4	<code>% This file deliberately does NOT run my_nlp.m, does NOT use result.mat, and</code>	Comment/documentation: This file deliberately does NOT run <code>my_nlp.m</code> , does NOT use <code>result.mat</code> , and
5	<code>% does NOT use any optimal-control solution. It only reuses:</code>	Comment/documentation: does NOT use any optimal-control solution. It only reuses:
6	<code>% - my_params.m</code>	Comment/documentation: - <code>my_params.m</code>
7	<code>% - my_model.m</code>	Comment/documentation: - <code>my_model.m</code>
8	<code>% - the same scaled state/control/load definitions</code>	Comment/documentation: - the same scaled state/control/load definitions
9	<code>% - the same y_xdot time-domain derivative</code>	Comment/documentation: - the same <code>y_xdot</code> time-domain derivative
10	<code>% - the same h_load tyre vertical-load algebraic equations</code>	Comment/documentation: - the same <code>h_load</code> tyre vertical-load algebraic equations
11	<code>%</code>	Comment spacer inside the file header or section.
12	<code>% Example:</code>	Comment/documentation: Example:
13	<code>% addpath('D:\casadi_3_7_2')</code>	Comment/documentation: <code>addpath('D:\casadi_3_7_2')</code>
14	<code>% addpath('D:\Applications\TOTPT-main\oc')</code>	Comment/documentation: <code>addpath('D:\Applications\TOTPT-main\oc')</code>
15	<code>% [sim_ode15s, sim_fixed] = run_oc_simple_time_sim_arbitrary('mild_sine');</code>	Comment/documentation: <code>[sim_ode15s, sim_fixed] = run_oc_simple_time_sim_arbitrary('mild_sine');</code>
16	<code>%</code>	Comment spacer inside the file header or section.
17	<code>% Optional short run:</code>	Comment/documentation: Optional short run:
18	<code>% opts = struct('t_end',3,'fixed_dt',1e-3,'do_plot',true);</code>	Comment/documentation: <code>opts = struct('t_end',3,'fixed_dt',1e-3,'do_plot',true);</code>
19	<code>% [sim_ode15s, sim_fixed] = run_oc_simple_time_sim_arbitrary('mild_sine',opts);</code>	Comment/documentation: <code>[sim_ode15s, sim_fixed] = run_oc_simple_time_sim_arbitrary('mild_sine',opts);</code>
20	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
21	<code>if nargin < 1 isempty(profile)</code>	Checks whether the user omitted the first input argument or passed an empty profile.
22	<code> profile = 'mild_sine';</code>	Uses <code>mild_sine</code> as the default arbitrary command profile.
23	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

Line	Source	Explanation
24	<code>if nargin < 2 isempty(user_opts)</code>	Checks whether the user omitted the options structure.
25	<code> user_opts = struct();</code>	Creates an empty structure so default_options can fill in all option fields.
26	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
27	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
28	<code>fprintf('\n=== Simple time simulation from my_model.m only ===\n');</code>	Prints a banner so you know this run is not using my_nlp, result.mat, or an optimal-control solution.
29	<code>fprintf('Profile: %s\n', profile);</code>	Executable MATLAB statement in Main driver: run_oc_simple_time_sim_arbitrary.
30	<code>fprintf('Simulator version: 2026-06-23-v3-checked-true- ode15i-dae.\n');</code>	Calls or configures MATLAB ode15i, a variable-step fully implicit DAE solver.
31	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
32	<code>model = build_model_functions_from_my_model();</code>	Builds a MATLAB structure containing vehicle parameters, scaling vectors, bounds, and CasADi functions generated from my_model.m.
33	<code>veh = model.veh;</code>	Copies the vehicle parameter structure out of model for shorter local calls.
34	<code>opts = default_options(user_opts, veh);</code>	Creates the simulator options and then overwrites any default fields with user_opts.
35	<code>ctrl = make_arbitrary_controls(profile, opts, veh);</code>	Builds the chosen arbitrary input histories: drive torque command, steering command, and zero brake command.
36	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
37	<code>x0_scaled = make_initial_state_scaled(veh, model);</code>	Creates the initial 13-state vector in scaled coordinates.
38	<code>z0_static_scaled = static_load_guess_scaled(veh, model);</code>	Creates a static vertical-load initial guess in scaled coordinates.
39	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
40	<code>% For the reduced-ODE integrators, improve z0 by solving h_load = 0 once.</code>	Comment/documentation: For the reduced-ODE integrators, improve z0 by solving h_load = 0 once.
41	<code>% For the true DAE integrator, use z0_static_scaled and let decic/ode15i</code>	Comment/documentation: For the true DAE integrator, use z0_static_scaled and let decic/ode15i
42	<code>% determine a consistent algebraic initial condition without calling our</code>	Comment/documentation: determine a consistent algebraic initial condition without calling our
43	<code>% tyre-load Newton solve.</code>	Comment/documentation: tyre-load Newton solve.
44	<code>u0_scaled = make_applied_control_scaled(ctrl.t(1), x0_scaled, ctrl, veh, model, opts);</code>	Evaluates the applied, scaled control at t=0, including saturation and power limiting.
45	<code>kappa0 = eval_kappa(opts, 0, x0_scaled);</code>	Evaluates the initial road curvature parameter. By default this is zero for a straight road.
46	<code>z0_reduced_scaled = solve_loads_newton(model, x0_scaled, u0_scaled, kappa0, z0_static_scaled, opts, true);</code>	Solves h_load=0 once to give the reduced-ODE integrators a consistent initial tyre-load vector.
47	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
48	<code>% Variable-step stiff integration using ode15s. This is an ODE in x only,</code>	Comment/documentation: Variable-step stiff integration using ode15s. This is an ODE in x only,
49	<code>% with the tyre-load algebraic equations solved inside the RHS.</code>	Comment/documentation: with the tyre-load algebraic equations solved inside the RHS.
50	<code>fprintf('Running ode15s variable-step stiff simulation...\n');</code>	Announces the first integrator: variable-step ode15s on the reduced ODE in x only.

Line	Source	Explanation
51	<code>t_ode_start = tic;</code>	Computes or assigns <code>t_ode_start</code> in Main driver: <code>run_oc_simple_time_sim_arbitrary</code> . The exact meaning follows from the variable name and the surrounding function.
52	<code>z_cache = z0_reduced_scaled;</code>	Initializes the algebraic-load cache. The cache is only an initial guess for the next nonlinear load solve.
53	<code>odefun = @(t,xs) rhs_scaled(t, xs, ctrl, veh, model, opts, z_cache);</code>	Defines <code>odefun</code> but never uses it later. This is harmless dead code; <code>rhs_cached</code> below is the function actually passed to <code>ode15s</code> .
54	<code>% Use nested wrapper to update the z cache after each accepted/internal call.</code>	Comment/documentation: Use nested wrapper to update the <code>z</code> cache after each accepted/internal call.
55	<code>function dxs = rhs_cached(t, xs)</code>	Defines a nested function so MATLAB can update the captured <code>z_cache</code> variable between RHS calls.
56	<code> [dxs, z_cache] = rhs_scaled(t, xs, ctrl, veh, model, opts, z_cache);</code>	Computes dx/dt and the newest algebraic loads. The returned <code>z</code> becomes the next Newton initial guess.
57	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
58	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
59	<code>tspan = 0:opts.output_dt:opts.t_end;</code>	Builds requested output times. For <code>ode15s</code> this is not the internal step grid; it is mainly the reporting/interpolation grid.
60	<code>ode_opts = odeset('RelTol', opts.ode_reltol, 'AbsTol', opts.ode_abstol, ...</code>	Starts the <code>ode15s</code> option structure with relative and absolute error tolerances.
61	<code> 'MaxStep', opts.ode_max_step);</code>	Limits the largest internal step <code>ode15s</code> is allowed to take.
62	<code>[t_ode, x_ode] = ode15s(@rhs_cached, tspan, x0_scaled, ode_opts);</code>	Runs <code>ode15s</code> on the reduced state vector <code>x</code> . The tyre loads are solved inside every RHS call.
63	<code>time_ode15s = toc(t_ode_start);</code>	Calls or configures MATLAB <code>ode15s</code> , a variable-step stiff ODE solver.
64	<code>fprintf(' ode15s wall time: %.3f s, output points: %d.\n', time_ode15s, numel(t_ode));</code>	Calls or configures MATLAB <code>ode15s</code> , a variable-step stiff ODE solver.
65	<code>sim_ode15s = postprocess_solution('ode15s', t_ode(:), x_ode.', ctrl, veh, model, opts, z0_reduced_scaled);</code>	Postprocesses the <code>ode15s</code> state trajectory and recomputes loads, controls, residuals, accelerations, torques, μ ratios, and load diagnostics.
66	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
67	<code>% Fixed-step RK4 integration.</code>	Comment/documentation: Fixed-step RK4 integration.
68	<code>fprintf('Running fixed-step RK4 simulation... fixed_dt = %.4g s, stages/step = 4.\n', opts.fixed_dt);</code>	Announces the fixed-step RK4 run and reports its user-specified step size.
69	<code>t_fixed_start = tic;</code>	Computes or assigns <code>t_fixed_start</code> in Main driver: <code>run_oc_simple_time_sim_arbitrary</code> . The exact meaning follows from the variable name and the surrounding function.
70	<code>sim_fixed = simulate_fixed_rk4(x0_scaled, z0_reduced_scaled, ctrl, veh, model, opts);</code>	Runs the handwritten fixed-step RK4 integrator.
71	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
72	<code>time_fixed = toc(t_fixed_start);</code>	Computes or assigns <code>time_fixed</code> in Main driver: <code>run_oc_simple_time_sim_arbitrary</code> . The exact meaning follows from the variable name and the surrounding function.
73	<code>fprintf(' fixed RK4 wall time: %.3f s, steps: %d, nominal RHS calls: %d.\n', time_fixed, numel(sim_fixed.t)-1, 4*(numel(sim_fixed.t)-1));</code>	Reports RK4 computational cost. Nominal RHS calls are 4 times the number of time steps.
74	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
75	<code>% True DAE integration using ode15i. Here the four tyre vertical loads are</code>	Comment/documentation: True DAE integration using <code>ode15i</code> . Here the four tyre vertical loads are

Line	Source	Explanation
76	% solver variables, not quantities solved by our RHS Newton reduction. MATLAB	Comment/documentation: solver variables, not quantities solved by our RHS Newton reduction. MATLAB
77	% will solve the coupled nonlinear DAE residual internally.	Comment/documentation: will solve the coupled nonlinear DAE residual internally.
78	if opts.run_dae	Only runs the true DAE simulation if opts.run_dae is true.
79	fprintf('Running ode15i true DAE simulation for [x; z]...\n');	Calls or configures MATLAB ode15i, a variable-step fully implicit DAE solver.
80	t_dae_start = tic;	Computes or assigns t_dae_start in Main driver: run_oc_simple_time_sim_arbitrary. The exact meaning follows from the variable name and the surrounding function.
81	sim_ode15i_dae = simulate_ode15i_dae(x0_scaled, z0_static_scaled, ctrl, veh, model, opts);	Runs ode15i on the coupled [x; z] DAE. This path does not call the user Newton load solver during the residual evaluation.
82	time_dae = toc(t_dae_start);	Computes or assigns time_dae in Main driver: run_oc_simple_time_sim_arbitrary. The exact meaning follows from the variable name and the surrounding function.
83	fprintf(' ode15i DAE wall time: %.3f s, output points: %d.\n', time_dae, numel(sim_ode15i_dae.t));	Calls or configures MATLAB ode15i, a variable-step fully implicit DAE solver.
84	else	Alternative branch of the preceding if/switch logic.
85	sim_ode15i_dae = [];	If the DAE run is disabled, the third return is an empty array.
86	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
87	[blank]	Blank line used to visually separate logical pieces of the code.
88	if isempty(sim_ode15i_dae)	Chooses the completion message depending on whether the DAE run exists.
89	fprintf('Done. Max h_load ode15s = %.3e, fixed = %.3e\n', ...	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
90	max(abs(sim_ode15s.h_load(:))), max(abs(sim_fixed.h_load(:))));	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
91	else	Alternative branch of the preceding if/switch logic.
92	fprintf('Done. Max h_load ode15s = %.3e, fixed = %.3e, ode15i DAE = %.3e\n', ...	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
93	max(abs(sim_ode15s.h_load(:))), max(abs(sim_fixed.h_load(:))), max(abs(sim_ode15i_dae.h_load(:))));	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
94	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
95	[blank]	Blank line used to visually separate logical pieces of the code.
96	if opts.do_plot	Only opens MATLAB figures if opts.do_plot is true.
97	plot_simple_results(sim_ode15s, sim_fixed, sim_ode15i_dae, ctrl, veh);	Plots speed, selected states, controls, tyre loads, residuals, and DAE-relative speed error if available.
98	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
99	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

build_model_functions_from_my_model (lines 102-137)

Line	Source	Explanation
102	<code>function model = build_model_functions_from_my_model()</code>	Begins the helper that converts my_model.m outputs into reusable single-output CasADi functions.
103	<code>% Build only single-output CasADi functions. No NLP, no collocation, no IPOPT.</code>	Comment/documentation: Build only single-output CasADi functions. No NLP, no collocation, no IPOPT.
104	<code>import casadi.*</code>	Imports CasADi symbols and Function constructor into this helper scope.
105	<code>veh = my_params(); %#ok<NASGU></code>	Loads the vehicle parameter structure by calling my_params(). The %#ok suppresses the local unused warning before run(my_model.m) uses veh.
106	<code>run('my_model.m');</code>	Executes my_model.m in this function workspace, so variables such as x, u, z, y_xdot, h_load, and scaling vectors become available.
107	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
108	<code>required = {'x','u','z','pv','y_xdot','h_load','y_acc','y_torque','y_mu','y_load', ...</code>	Lists all symbols and arrays that must exist after running my_model.m.
109	<code> 'x_s','u_s','z_s','x_min','x_max','z_min','z_max','nx','nz'};</code>	Executable MATLAB statement in build_model_functions_from_my_model.
110	<code>for i = 1:numel(required)</code>	Loops through the required list to validate that my_model.m created every needed object.
111	<code> assert(exist(required{i}, 'var') == 1, 'my_model.m did not define %s.', required{i});</code>	Stops immediately with a useful error message if a required model variable is missing.
112	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
113	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
114	<code>model = struct();</code>	Creates the model wrapper structure returned to the caller.
115	<code>model.veh = veh;</code>	Computes or assigns model.veh in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
116	<code>model.nx = nx;</code>	Computes or assigns model.nx in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
117	<code>model.nz = nz;</code>	Computes or assigns model.nz in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
118	<code>model.x_s = full(x_s(:));</code>	Computes or assigns model.x_s in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
119	<code>model.u_s = full(u_s(:));</code>	Computes or assigns model.u_s in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
120	<code>model.z_s = full(z_s(:));</code>	Computes or assigns model.z_s in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
121	<code>model.x_min = full(x_min(:));</code>	Computes or assigns model.x_min in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
122	<code>model.x_max = full(x_max(:));</code>	Computes or assigns model.x_max in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
123	<code>model.z_min = full(z_min(:));</code>	Computes or assigns model.z_min in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
124	<code>model.z_max = full(z_max(:));</code>	Computes or assigns model.z_max in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
125	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.

Line	Source	Explanation
126	% Single-output functions only. This avoids the previous multi-output and	Comment/documentation: Single-output functions only. This avoids the previous multi-output and
127	% concatenated-output issues completely.	Comment/documentation: concatenated-output issues completely.
128	model.f_xdot = casadi.Function('sim_f_xdot', {x,u,z,pv}, {y_xdot});	Creates a CasADi function for the time-domain scaled state derivative y_xdot(x,u,z,kappa).
129	model.f_hload = casadi.Function('sim_f_hload', {x,u,z,pv}, {h_load});	Creates a CasADi function for the four scaled tyre-load residuals h_load(x,u,z,kappa).
130	model.f_hload_jac_z = casadi.Function('sim_f_hload_jac_z', {x,u,z,pv}, {jacobian(h_load, z)});	Creates the analytic 4-by-4 Jacobian dh_load/dz used by the Newton load solver.
131	model.f_acc = casadi.Function('sim_f_acc', {x,u,z,pv}, {y_acc});	Creates a diagnostic acceleration function.
132	model.f_torque = casadi.Function('sim_f_torque', {x,u,z,pv}, {y_torque});	Creates a diagnostic wheel-torque function.
133	model.f_mu = casadi.Function('sim_f_mu', {x,u,z,pv}, {y_mu});	Creates a diagnostic tyre-utilization function.
134	model.f_load = casadi.Function('sim_f_load', {x,u,z,pv}, {y_load});	Creates a diagnostic load output function containing residuals and Chapter-9 load components.
135	[blank]	Blank line used to visually separate logical pieces of the code.
136	fprintf('Built CasADi functions from my_model.m. nx = %d, nz = %d.\n', model.nx, model.nz);	Computes or assigns fprintf('Built CasADi functions from my_model.m. nx in build_model_functions_from_my_model. The exact meaning follows from the variable name and the surrounding function.
137	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

default_options (lines 140-168)

Line	Source	Explanation
140	function opts = default_options(user_opts, veh)	Begins the default option constructor.
141	opts = struct();	Creates an empty MATLAB structure that will be filled with fields.
142	opts.t_end = 15.0;	Default simulation duration is 15 seconds.
143	opts.output_dt = 0.02;	Default output spacing is 0.02 seconds. It also sets the command-grid spacing used by make_arbitrary_controls.
144	opts.fixed_dt = 1e-3;	Default fixed RK4 internal step size is 0.001 seconds.
145	opts.ode_reltol = 1e-6;	Default ode15s relative tolerance.
146	opts.ode_abstol = 1e-8;	Default ode15s absolute tolerance.
147	opts.ode_max_step = 0.02;	Default maximum internal ode15s step.
148	opts.dae_reltol = 1e-6;	Default ode15i relative tolerance.
149	opts.dae_abstol = 1e-8;	Default ode15i absolute tolerance.
150	opts.dae_max_step = 0.02;	Default maximum internal ode15i step.
151	opts.do_plot = true;	Default behavior is to generate plots.
152	opts.run_dae = true;	Default behavior is to run the third, true DAE integrator.

Line	Source	Explanation
153	% Fixed RK4 should not silently clamp states by default, because ode15s and	Comment/documentation: Fixed RK4 should not silently clamp states by default, because ode15s and
154	% ode15i do not impose the NLP state bounds during simulation. Set this true	Comment/documentation: ode15i do not impose the NLP state bounds during simulation. Set this true
155	% only as a numerical guard for aggressive arbitrary inputs.	Comment/documentation: only as a numerical guard for aggressive arbitrary inputs.
156	opts.enforce_state_bounds_fixed = false;	Default is no RK4 state clamping, so RK4, ode15s, and ode15i are solving the same simulation model.
157	opts.kappa_constant = 0.0; % straight road by default	Default road curvature is zero, i.e. straight road.
158	opts.load_tol = 1e-8;	Newton load-solve convergence tolerance on scaled h_load.
159	opts.load_max_iter = 12;	Maximum Newton iterations for each tyre-load solve.
160	opts.load_fd_rel = 1e-6;	Legacy finite-difference option. In this version the analytic CasADi Jacobian is used, so this field is effectively unused.
161	opts.warn_load_residual = 1e-5;	Threshold above which a tyre-load residual warning is issued.
162	opts.min_power_omega = max(1.0, veh.omega_rear_axle_min);	Minimum wheel speed used when applying the motor power limit to avoid dividing by a near-zero speed.
163	[blank]	Blank line used to visually separate logical pieces of the code.
164	names = fieldnames(user_opts);	Gets all user-supplied option names.
165	for i = 1:numel(names)	Starts a loop in default_options.
166	opts.(names{i}) = user_opts.(names{i});	Overwrites defaults with user choices field by field.
167	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
168	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

make_arbitrary_controls (lines 171-194)

Line	Source	Explanation
171	function ctrl = make_arbitrary_controls(profile, opts, veh)	Begins the arbitrary command-profile generator.
172	t = 0:opts.output_dt:opts.t_end;	Creates the command grid. This grid spacing is opts.output_dt, not RK4 fixed_dt.
173	Tt_max = veh.Tt_max;	Reads the maximum drive torque from vehicle parameters.
174	[blank]	Blank line used to visually separate logical pieces of the code.
175	switch lower(profile)	Selects which command profile to generate after converting the profile string to lowercase.
176	case 'straight_accel'	straight_accel profile: torque ramp with zero steering.
177	Tt = 0.45*Tt_max*(1 - exp(-t/2.0));	Computes or assigns Tt in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.
178	delta = zeros(size(t));	Preallocates an array of zeros for efficiency and predictable dimensions.
179	case 'mild_sine'	mild_sine profile: modest torque ramp plus small sinusoidal steering.
180	Tt = 0.35*Tt_max*(1 - exp(-t/2.0)).*(1 + 0.08*sin(2*pi*0.20*t));	Computes or assigns Tt in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.

Line	Source	Explanation
181	<code>delta = deg2rad(3.0)*sin(2*pi*0.18*t).*(1 - exp(-t/1.0));</code>	Computes or assigns delta in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.
182	<code>case 'slalom'</code>	slalom profile: smaller torque ramp plus larger, faster sinusoidal steering.
183	<code>Tt = 0.25*Tt_max*(1 - exp(-t/1.5));</code>	Computes or assigns Tt in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.
184	<code>delta = deg2rad(6.0)*sin(2*pi*0.35*t).*(1 - exp(-t/1.0));</code>	Computes or assigns delta in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.
185	<code>otherwise</code>	Fallback switch branch when no named case matches.
186	<code>error('Unknown profile "%s". Use straight_accel, mild_sine, or slalom.', profile);</code>	Rejects unknown profile names with a clear error message.
187	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
188	[blank]	Blank line used to visually separate logical pieces of the code.
189	<code>ctrl = struct();</code>	Creates the control history structure.
190	<code>ctrl.t = t(:).';</code>	Computes or assigns ctrl.t in make_arbitrary_controls. The exact meaning follows from the variable name and the surrounding function.
191	<code>ctrl.Tt_cmd = min(max(Tt(:).', veh.Tt_min), veh.Tt_max);</code>	Saturates drive torque commands to vehicle torque bounds.
192	<code>ctrl.delta_cmd = min(max(delta(:).', veh.delta_min), veh.delta_max);</code>	Saturates steering commands to vehicle steering bounds.
193	<code>ctrl.Tb_cmd = zeros(size(ctrl.t)); % brake torque fixed zero as requested</code>	Stores a zero brake history. Note that make_applied_control_scaled later hard-codes Tb=0, so this field is currently only informational.
194	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

make_initial_state_scaled (lines 197-215)

Line	Source	Explanation
197	<code>function x0_scaled = make_initial_state_scaled(veh, model)</code>	Begins construction of the initial scaled state.
198	<code>ux0 = 5.0;</code>	Initial longitudinal speed is 5 m/s.
199	<code>x0_phys = [...</code>	Computes or assigns x0_phys in make_initial_state_scaled. The exact meaning follows from the variable name and the surrounding function.
200	<code>ux0; % ux</code>	Executable MATLAB statement in make_initial_state_scaled.
201	<code>0; % uy</code>	Executable MATLAB statement in make_initial_state_scaled.
202	<code>0; % r</code>	Executable MATLAB statement in make_initial_state_scaled.
203	<code>0; % phi</code>	Executable MATLAB statement in make_initial_state_scaled.
204	<code>0; % phidot</code>	Executable MATLAB statement in make_initial_state_scaled.
205	<code>0; % theta</code>	Executable MATLAB statement in make_initial_state_scaled.
206	<code>0; % thetadot</code>	Executable MATLAB statement in make_initial_state_scaled.
207	<code>0; % n</code>	Executable MATLAB statement in make_initial_state_scaled.
208	<code>0; % xi</code>	Executable MATLAB statement in make_initial_state_scaled.

Line	Source	Explanation
209	ux0/veh.rw; % omega_fl	Executable MATLAB statement in make_initial_state_scaled.
210	ux0/veh.rw; % omega_fr	Executable MATLAB statement in make_initial_state_scaled.
211	ux0/veh.rw; % omega_rl	Executable MATLAB statement in make_initial_state_scaled.
212	ux0/veh.rw]; % omega_rr	Executable MATLAB statement in make_initial_state_scaled.
213	x0_scaled = x0_phys(:) ./ model.x_s(:);	Converts physical initial states to scaled states by dividing by x_s.
214	x0_scaled = min(max(x0_scaled, model.x_min), model.x_max);	Clamps initial states into my_model.m state bounds.
215	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

static_load_guess_scaled (lines 218-224)

Line	Source	Explanation
218	function z0_scaled = static_load_guess_scaled(veh, model)	Begins static vertical-load initial guess construction.
219	Z_front = veh.m*veh.g*veh.lr/veh.l;	Computes the total static front axle load from static weight distribution.
220	Z_rear = veh.m*veh.g*veh.lf/veh.l;	Computes the total static rear axle load from static weight distribution.
221	z_phys = [0.5*Z_front; 0.5*Z_front; 0.5*Z_rear; 0.5*Z_rear];	Splits each axle load equally left-right.
222	z0_scaled = z_phys ./ model.z_s(:);	Converts physical loads to scaled algebraic variables.
223	z0_scaled = min(max(z0_scaled, model.z_min), model.z_max);	Clamps the load guess into allowed load bounds.
224	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

rhs_scaled (lines 227-234)

Line	Source	Explanation
227	function [dxs, z_new] = rhs_scaled(t, xs, ctrl, veh, model, opts, z_guess)	Begins the reduced-ODE RHS. It returns scaled state derivatives and the current solved load vector.
228	xs = xs(:);	Forces the state vector into a column vector.
229	kappa = eval_kappa(opts, t, xs);	Evaluates road curvature for the current time and state.
230	u_scaled = make_applied_control_scaled(t, xs, ctrl, veh, model, opts);	Builds the currently applied scaled control after interpolation, saturation, and power limiting.
231	z_new = solve_loads_newton(model, xs, u_scaled, kappa, z_guess, opts, false);	Solves the algebraic tyre-load equations h_load=0 before evaluating the state derivative.
232	dxs = full(model.f_xdot(xs, u_scaled, z_new, kappa));	Evaluates the model time derivative using the consistent load vector.
233	dxs = dxs(:);	Forces the derivative into a column vector for MATLAB ODE solvers.
234	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

make_applied_control_scaled (lines 237-253)

Line	Source	Explanation
237	<code>function u_scaled = make_applied_control_scaled(t, xs, ctrl, veh, model, opts)</code>	Begins conversion from command histories to applied scaled controls.
238	<code>Tt_cmd = interp1(ctrl.t, ctrl.Tt_cmd, t, 'linear', 'extrap');</code>	Linearly interpolates drive torque command at the requested time.
239	<code>delta_cmd = interp1(ctrl.t, ctrl.delta_cmd, t, 'linear', 'extrap');</code>	Linearly interpolates steering command at the requested time.
240	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
241	<code>omega_rl = xs(12) * model.x_s(12);</code>	Converts scaled rear-left wheel speed to physical rad/s.
242	<code>omega_rr = xs(13) * model.x_s(13);</code>	Converts scaled rear-right wheel speed to physical rad/s.
243	<code>omega_rear = 0.5*(omega_rl + omega_rr);</code>	Averages rear wheel speeds for the motor power limit calculation.
244	<code>omega_for_power = max(abs(omega_rear), opts.min_power_omega);</code>	Protects the power-limit division by enforcing a minimum speed magnitude.
245	<code>Tt_power_max = veh.power_motor_max / omega_for_power;</code>	Converts power limit $P_{\max}$ into a torque limit $T=P/\omega$.
246	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
247	<code>Tt = min(max(Tt_cmd, veh.Tt_min), min(veh.Tt_max, Tt_power_max));</code>	Applies lower torque bound, upper torque bound, and power-limit torque cap.
248	<code>Tb = 0.0;</code>	Brake torque is hard-coded to zero in the applied controls.
249	<code>delta = min(max(delta_cmd, veh.delta_min), veh.delta_max);</code>	Applies steering bounds to the command.
250	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
251	<code>u_phys = [Tt; Tb; delta];</code>	Packs physical controls in the order [drive torque; brake torque; steering].
252	<code>u_scaled = u_phys ./ model.u_s(:);</code>	Scales the physical controls by u_s for the CasADi model.
253	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

eval_kappa (lines 256-264)

Line	Source	Explanation
256	<code>function kappa = eval_kappa(opts, t, xs) %#ok<INUSD></code>	Begins the curvature evaluator.
257	<code>% The simple simulator uses a straight road by default. You can replace this</code>	Comment/documentation: The simple simulator uses a straight road by default. You can replace this
258	<code>% with a function handle through opts.kappa_fun if needed.</code>	Comment/documentation: with a function handle through opts.kappa_fun if needed.
259	<code>if isfield(opts, 'kappa_fun') && ~isempty(opts.kappa_fun)</code>	Uses a user-supplied curvature function if opts.kappa_fun exists and is not empty.
260	<code> kappa = opts.kappa_fun(t, xs);</code>	Calls the user curvature function with current time and scaled state.
261	<code>else</code>	Alternative branch of the preceding if/switch logic.
262	<code> kappa = opts.kappa_constant;</code>	Otherwise returns the constant curvature value, zero by default.
263	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
264	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

solve_loads_newton (lines 267-336)

Line	Source	Explanation
267	function z = solve_loads_newton(model, xs, us, kappa, z_guess, opts, do_warn)	Begins the Newton solver for the four algebraic tyre-load variables.
268	% Solve the four lifted tyre-load equations h_load(x,u,z,kappa)=0.	Comment/documentation: Solve the four lifted tyre-load equations h_load(x,u,z,kappa)=0.
269	%	Comment spacer inside the file header or section.
270	% This is the core DAE reduction used by both integrators. z is not	Comment/documentation: This is the core DAE reduction used by both integrators. z is not
271	% integrated as a state; at every RHS/stage evaluation it is recomputed so	Comment/documentation: integrated as a state; at every RHS/stage evaluation it is recomputed so
272	% that the exact same Chapter-9 load residuals from my_model.m are satisfied.	Comment/documentation: that the exact same Chapter-9 load residuals from my_model.m are satisfied.
273	%	Comment spacer inside the file header or section.
274	% Version v2 uses the analytic CasADi Jacobian dh_load/dz instead of finite	Comment/documentation: Version v2 uses the analytic CasADi Jacobian dh_load/dz instead of finite
275	% differencing the four algebraic variables. This is much faster for RK4,	Comment/documentation: differencing the four algebraic variables. This is much faster for RK4,
276	% because RK4 evaluates the RHS four times per fixed step.	Comment/documentation: because RK4 evaluates the RHS four times per fixed step.
277	z = z_guess(:);	Starts Newton from the supplied guess as a column vector.
278	z = min(max(z, model.z_min), model.z_max);	Clamps the starting load guess into load bounds.
279	[blank]	Blank line used to visually separate logical pieces of the code.
280	r = full(model.f_hload(xs, us, z, kappa));	Evaluates the current load residual h_load.
281	r = r(:);	Computes or assigns r in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
282	if norm(r, inf) < opts.load_tol	If the initial guess already satisfies the tolerance, it returns immediately.
283	return;	Executable MATLAB statement in solve_loads_newton.
284	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
285	[blank]	Blank line used to visually separate logical pieces of the code.
286	for iter = 1:opts.load_max_iter	Starts the damped Newton iteration loop.
287	r = full(model.f_hload(xs, us, z, kappa));	Evaluates a CasADi function and converts the result to a full MATLAB numeric array.
288	r = r(:);	Computes or assigns r in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
289	rnorm = norm(r, inf);	Computes or assigns rnorm in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
290	if rnorm < opts.load_tol	Starts a conditional branch in solve_loads_newton.
291	return;	Executable MATLAB statement in solve_loads_newton.
292	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
293	[blank]	Blank line used to visually separate logical pieces of the code.
294	J = full(model.f_hload_jac_z(xs, us, z, kappa));	Evaluates the analytic Jacobian dh_load/dz from CasADi.
295	if any(~isfinite(J(:))) any(~isfinite(r(:)))	Starts a conditional branch in solve_loads_newton.

Line	Source	Explanation
296	break;	Executable MATLAB statement in solve_loads_newton.
297	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
298	[blank]	Blank line used to visually separate logical pieces of the code.
299	% Four-by-four Newton system. Use pinv only as a last-resort guard.	Comment/documentation: Four-by-four Newton system. Use pinv only as a last-resort guard.
300	if rcond(J) < 1e-12	Checks whether the 4-by-4 Jacobian is nearly singular.
301	step = -pinv(J) * r;	Uses pseudo-inverse only as a last-resort safeguard for a nearly singular Jacobian.
302	else	Alternative branch of the preceding if/switch logic.
303	step = -J \ r;	Uses the standard Newton step for a well-conditioned Jacobian.
304	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
305	if any(~isfinite(step))	Starts a conditional branch in solve_loads_newton.
306	break;	Executable MATLAB statement in solve_loads_newton.
307	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
308	[blank]	Blank line used to visually separate logical pieces of the code.
309	% Damped Newton with simple backtracking and z-bounds preservation.	Comment/documentation: Damped Newton with simple backtracking and z-bounds preservation.
310	alpha = 1.0;	Initial Newton damping factor.
311	accepted = false;	Computes or assigns accepted in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
312	for bt = 1:8	Allows up to eight backtracking halvings of the Newton step.
313	z_trial = z + alpha*step;	Tests a damped Newton trial load vector.
314	z_trial = min(max(z_trial, model.z_min), model.z_max);	Preserves load bounds during the trial.
315	r_trial = full(model.f_hload(xs, us, z_trial, kappa));	Evaluates the residual at the trial load vector.
316	r_trial = r_trial(:);	Computes or assigns r_trial in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
317	if norm(r_trial, inf) <= 0.9*rnorm norm(r_trial, inf) < opts.load_tol	Accepts the trial if it reduces residual by at least 10 percent or reaches tolerance.
318	z = z_trial;	Computes or assigns z in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
319	accepted = true;	Computes or assigns accepted in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
320	break;	Executable MATLAB statement in solve_loads_newton.
321	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
322	alpha = 0.5*alpha;	Halves the step if the trial was not accepted.
323	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
324	[blank]	Blank line used to visually separate logical pieces of the code.
325	if ~accepted	If all backtracking failed, takes a guarded small step instead of a full bad Newton step.

Line	Source	Explanation
326	<code>% Do not take a full bad Newton step; take a guarded small step.</code>	Comment/documentation: Do not take a full bad Newton step; take a guarded small step.
327	<code>z = min(max(z + 0.10*step, model.z_min), model.z_max);</code>	Applies lower and upper saturation/clamping bounds.
328	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
329	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
330	[blank]	Blank line used to visually separate logical pieces of the code.
331	<code>r = full(model.f_hload(xs, us, z, kappa));</code>	Evaluates the final residual after the iteration loop.
332	<code>rnorm = norm(r(:), inf);</code>	Computes or assigns rnorm in solve_loads_newton. The exact meaning follows from the variable name and the surrounding function.
333	<code>if do_warn rnorm > opts.warn_load_residual</code>	Warns if requested or if the final residual exceeds the warning threshold.
334	<code>warning('Tyre-load Newton residual max h_load = %3e.', rnorm);</code>	Issues a MATLAB warning rather than stopping execution.
335	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
336	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

simulate_fixed_rk4 (lines 339-366)

Line	Source	Explanation
339	<code>function sim = simulate_fixed_rk4(x0_scaled, z0_scaled, ctrl, veh, model, opts)</code>	Begins the handwritten fixed-step RK4 simulator.
340	<code>dt = opts.fixed_dt;</code>	Reads the RK4 step size from opts.fixed_dt.
341	<code>t = 0:dt:opts.t_end;</code>	Builds the actual RK4 time grid. This is the fixed internal grid, not just output times.
342	<code>X = zeros(model.nx, numel(t));</code>	Preallocates the scaled state trajectory.
343	<code>Z = zeros(model.nz, numel(t));</code>	Preallocates the scaled load trajectory storage.
344	<code>X(:,1) = x0_scaled(:);</code>	Stores the initial scaled state.
345	<code>Z(:,1) = z0_scaled(:);</code>	Stores the initial load guess.
346	<code>z_guess = z0_scaled(:);</code>	Initializes the load guess cache used by the first RK4 stage.
347	[blank]	Blank line used to visually separate logical pieces of the code.
348	<code>for k = 1:numel(t)-1</code>	Loops over fixed time steps.
349	<code>tk = t(k);</code>	Computes or assigns tk in simulate_fixed_rk4. The exact meaning follows from the variable name and the surrounding function.
350	<code>xk = X(:,k);</code>	Computes or assigns xk in simulate_fixed_rk4. The exact meaning follows from the variable name and the surrounding function.
351	[blank]	Blank line used to visually separate logical pieces of the code.
352	<code>[k1, z1] = rhs_scaled(tk, xk, ctrl, veh, model, opts, z_guess);</code>	RK4 stage 1: evaluates RHS at the beginning of the step and solves loads there.
353	<code>[k2, z2] = rhs_scaled(tk + 0.5*dt, xk + 0.5*dt*k1, ctrl, veh, model, opts, z1);</code>	RK4 stage 2: evaluates RHS at the midpoint predicted from k1 and solves loads there.

Line	Source	Explanation
354	<code>[k3, z3] = rhs_scaled(tk + 0.5*dt, xk + 0.5*dt*k2, ctrl, veh, model, opts, z2);</code>	RK4 stage 3: evaluates RHS at the midpoint predicted from k2 and solves loads there.
355	<code>[k4, z4] = rhs_scaled(tk + dt, xk + dt*k3, ctrl, veh, model, opts, z3);</code>	RK4 stage 4: evaluates RHS at the end predicted from k3 and solves loads there.
356	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
357	<code>X(:,k+1) = xk + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);</code>	Applies the classical fourth-order RK4 weighted state update.
358	<code>if opts.enforce_state_bounds_fixed</code>	Optionally enforces state bounds only for RK4.
359	<code> X(:,k+1) = min(max(X(:,k+1), model.x_min), model.x_max);</code>	Applies lower and upper saturation/clamping bounds.
360	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
361	<code>Z(:,k+1) = z4;</code>	Stores the final stage load vector as the representative load at the new time point.
362	<code>z_guess = z4;</code>	Uses the final stage load as the next step Newton initial guess.
363	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
364	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
365	<code>sim = postprocess_solution('fixed_rk4', t(:), X, ctrl, veh, model, opts, z0_scaled);</code>	Postprocesses the RK4 trajectory on its fixed grid.
366	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

simulate_ode15i_dae (lines 369-430)

Line	Source	Explanation
369	<code>function sim = simulate_ode15i_dae(x0_scaled, z0_scaled, ctrl, veh, model, opts)</code>	Begins the true ode15i DAE simulator.
370	<code>%SIMULATE_ODE15I_DAE True index-1 DAE integration of [x; z].</code>	Comment/documentation: SIMULATE_ODE15I_DAE True index-1 DAE integration of [x; z].
371	<code>%</code>	Comment spacer inside the file header or section.
372	<code>% Residual form for ode15i:</code>	Comment/documentation: Residual form for ode15i:
373	<code>% 0 = xdot - f_xdot(x,u,z,kappa)</code>	Comment/documentation: $0 = \dot{x} - f_{\dot{x}}(x,u,z,kappa)$
374	<code>% 0 = h_load(x,u,z,kappa)</code>	Comment/documentation: $0 = h_{load}(x,u,z,kappa)$
375	<code>%</code>	Comment spacer inside the file header or section.
376	<code>% Unlike ode15s/RK4 above, this function does not call solve_loads_newton in</code>	Comment/documentation: Unlike ode15s/RK4 above, this function does not call solve_loads_newton in
377	<code>% the RHS. The four lifted tyre loads z are part of the solver state vector.</code>	Comment/documentation: the RHS. The four lifted tyre loads z are part of the solver state vector.
378	<code>% MATLAB's DAE solver handles the nonlinear algebraic coupling internally.</code>	Comment/documentation: MATLAB's DAE solver handles the nonlinear algebraic coupling internally.
379	<code>% A consistent initial condition is found by decic, with x(0) fixed and z(0)</code>	Comment/documentation: A consistent initial condition is found by decic, with $x(0)$ fixed and $z(0)$
380	<code>% allowed to move from the static-load guess.</code>	Comment/documentation: allowed to move from the static-load guess.

Line	Source	Explanation
381	[blank]	Blank line used to visually separate logical pieces of the code.
382	t0 = 0.0;	Sets initial time.
383	y0_guess = [x0_scaled(:); z0_scaled(:)];	Packs differential states and algebraic load variables into one DAE vector y=[x;z].
384	yp0_guess = zeros(model.nx + model.nz, 1);	Initial derivative guess for all DAE variables.
385	[blank]	Blank line used to visually separate logical pieces of the code.
386	% Give decic a good derivative guess for the differential states. This does	Comment/documentation: Give decic a good derivative guess for the differential states. This does
387	% not solve h_load manually; it only improves decic's starting point.	Comment/documentation: not solve h_load manually; it only improves decic's starting point.
388	u0 = make_applied_control_scaled(t0, x0_scaled, ctrl, veh, model, opts);	Computes the initial applied control for the derivative guess.
389	k0 = eval_kappa(opts, t0, x0_scaled);	Computes initial curvature for the derivative guess.
390	try	Starts a protected block; errors are caught by the following catch.
391	yp0_guess(1:model.nx) = full(model.f_xdot(x0_scaled(:), u0, z0_scaled(:), k0));	Uses f_xdot to give decic a good initial xdot guess.
392	catch	Fallback block if the preceding try statement fails.
393	yp0_guess(1:model.nx) = 0;	Falls back to zero derivative guess if that evaluation fails.
394	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
395	[blank]	Blank line used to visually separate logical pieces of the code.
396	fixed_y0 = [true(model.nx,1); false(model.nz,1)]; % keep physical initial x fixed; let loads become consistent	Tells decic to keep x(0) fixed but allow z(0) to move to consistency.
397	fixed_yp0 = [false(model.nx,1); true(model.nz,1)]; % z_dot is absent from residual, so fix it to zero	Tells decic that z_dot is fixed to zero because z_dot does not appear in the residual.
398	[blank]	Blank line used to visually separate logical pieces of the code.
399	resfun = @(t,y,yp) dae_residual_scaled(t, y, yp, ctrl, veh, model, opts);	Creates the residual function handle required by decic and ode15i.
400	decic_opts = odeset('RelTol', opts.dae_reltol, 'AbsTol', opts.dae_abstol);	Builds decic tolerances from DAE tolerance options.
401	[y0_cons, yp0_cons] = decic(resfun, t0, y0_guess, fixed_y0, yp0_guess, fixed_yp0, decic_opts);	Finds consistent initial y and ydot for the DAE.
402	res0_cons = resfun(t0, y0_cons, yp0_cons);	Evaluates the residual at the consistent initial condition.
403	res0_cons_norm = norm(res0_cons, inf);	Computes the infinity norm diagnostic of the initial residual.
404	if res0_cons_norm > 100*opts.dae_abstol	Warns if decic did not reduce the initial residual sufficiently.
405	warning('decic returned a relatively large initial DAE residual: %3e.', res0_cons_norm);	Issues a MATLAB warning rather than stopping execution.
406	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
407	[blank]	Blank line used to visually separate logical pieces of the code.
408	% Integrate the DAE. ode15i uses variable steps internally; tspan below only	Comment/documentation: Integrate the DAE. ode15i uses variable steps internally; tspan below only

Line	Source	Explanation
409	% fixes the requested output times for easier comparison with the other runs.	Comment/documentation: fixes the requested output times for easier comparison with the other runs.
410	tspan = 0:opts.output_dt:opts.t_end;	Builds requested DAE output times. ode15i still chooses variable internal steps.
411	dae_opts = odeset('RelTol', opts.dae_reltol, 'AbsTol', opts.dae_abstol, ...	Starts ode15i options with relative and absolute tolerances.
412	'MaxStep', opts.dae_max_step);	Limits the maximum internal DAE step size.
413	[t_dae, y_dae] = ode15i(resfun, tspan, y0_cons, yp0_cons, dae_opts);	Runs MATLAB ode15i on the implicit DAE residual.
414	[blank]	Blank line used to visually separate logical pieces of the code.
415	Y = y_dae.');	Transposes ode15i output so columns are time samples, matching the rest of the code.
416	X = Y(1:model.nx,:);	Extracts the differential state block X from the combined DAE solution.
417	Z = Y(model.nx+1:model.nx+model.nz,:);	Extracts the algebraic load block Z from the combined DAE solution.
418	[blank]	Blank line used to visually separate logical pieces of the code.
419	% Do not re-solve tyre loads here. Use the algebraic variables returned by	Comment/documentation: Do not re-solve tyre loads here. Use the algebraic variables returned by
420	% ode15i so that the diagnostic residual reflects the true DAE solution.	Comment/documentation: ode15i so that the diagnostic residual reflects the true DAE solution.
421	sim = postprocess_dae_solution('ode15i_DAE', t_dae(:), X, Z, ctrl, veh, model, opts);	Postprocesses using the DAE-returned loads without re-solving the algebraic equations.
422	[blank]	Blank line used to visually separate logical pieces of the code.
423	% Useful diagnostics for users comparing the reduced-ODE and true-DAE paths.	Comment/documentation: Useful diagnostics for users comparing the reduced-ODE and true-DAE paths.
424	sim.y0_guess = y0_guess;	Stores the original initial y guess for diagnostics.
425	sim.y0_consistent = y0_cons;	Stores the decic-consistent initial y.
426	sim.yp0_consistent = yp0_cons;	Stores the decic-consistent initial ydot.
427	sim.decic_residual_inf = res0_cons_norm;	Stores the initial DAE residual norm for checking consistency.
428	sim.initial_z_shift_scaled = y0_cons(model.nx+1:end) - z0_scaled(:);	Stores how much decic changed the scaled initial loads.
429	sim.initial_z_shift_phys = sim.initial_z_shift_scaled .* model.z_s(:);	Stores the same initial load shift in Newtons.
430	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

dae_residual_scaled (lines 433-447)

Line	Source	Explanation
433	function res = dae_residual_scaled(t, y, yp, ctrl, veh, model, opts)	Begins the implicit residual function used by ode15i.
434	y = y(:);	Forces y into a column vector.
435	yp = yp(:);	Forces yp into a column vector.

Line	Source	Explanation
436	<code>xs = y(1:model.nx);</code>	Extracts x from the combined DAE state.
437	<code>zs = y(model.nx+1:model.nx+model.nz);</code>	Extracts z from the combined DAE state.
438	<code>xsdot = yp(1:model.nx);</code>	Extracts xdot from the combined derivative vector.
439	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
440	<code>kap = eval_kappa(opts, t, xs);</code>	Evaluates curvature for the residual.
441	<code>us = make_applied_control_scaled(t, xs, ctrl, veh, model, opts);</code>	Evaluates applied control for the residual.
442	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
443	<code>f = full(model.f_xdot(xs, us, zs, kap));</code>	Evaluates the model state derivative f(x,u,z,kappa).
444	<code>h = full(model.f_hload(xs, us, zs, kap));</code>	Evaluates the algebraic load residual h_load(x,u,z,kappa).
445	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
446	<code>res = [xsdot(:) - f(:); h(:)];</code>	Returns the full DAE residual: differential residuals followed by algebraic load residuals.
447	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

postprocess_solution (lines 450-491)

Line	Source	Explanation
450	<code>function sim = postprocess_solution(name, t, X, ctrl, veh, model, opts, z0_scaled)</code>	Begins postprocessing for reduced-ODE solutions, where z must be reconstructed by Newton at each saved time.
451	<code>N = numel(t);</code>	Number of saved time samples.
452	<code>Z = zeros(model.nz, N);</code>	Preallocates reconstructed load trajectory.
453	<code>U = zeros(3, N);</code>	Preallocates applied control history.
454	<code>H = zeros(model.nz, N);</code>	Preallocates load residual diagnostics.
455	<code>ACC = zeros(7, N);</code>	Preallocates acceleration diagnostics.
456	<code>TORQUE = zeros(4, N);</code>	Preallocates wheel torque diagnostics.
457	<code>MU = zeros(4, N);</code>	Preallocates tyre utilization diagnostics.
458	<code>LOAD = zeros(11, N);</code>	Preallocates load-output diagnostics.
459	<code>z_guess = z0_scaled(:);</code>	Initializes the load Newton guess for postprocessing.
460	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
461	<code>for k = 1:N</code>	Loops through saved output samples.
462	<code> xs = X(:,k);</code>	Computes or assigns xs in postprocess_solution. The exact meaning follows from the variable name and the surrounding function.
463	<code> kap = eval_kappa(opts, t(k), xs);</code>	Computes or assigns kap in postprocess_solution. The exact meaning follows from the variable name and the surrounding function.
464	<code> us = make_applied_control_scaled(t(k), xs, ctrl, veh, model, opts);</code>	Computes or assigns us in postprocess_solution. The exact meaning follows from the variable name and the surrounding function.
465	<code> zs = solve_loads_newton(model, xs, us, kap, z_guess, opts, false);</code>	Solves the algebraic loads at this saved state for reduced-ODE postprocessing.

Line	Source	Explanation
466	<code>z_guess = zs;</code>	Computes or assigns <code>z_guess</code> in <code>postprocess_solution</code> . The exact meaning follows from the variable name and the surrounding function.
467	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
468	<code>Z(:,k) = zs;</code>	Stores solved scaled loads.
469	<code>U(:,k) = us;</code>	Stores applied scaled controls.
470	<code>H(:,k) = full(model.f_hload(xs, us, zs, kap));</code>	Stores the load residual after solving.
471	<code>ACC(:,k) = full(model.f_acc(xs, us, zs, kap));</code>	Stores acceleration outputs from <code>my_model.m</code> .
472	<code>TORQUE(:,k) = full(model.f_torque(xs, us, zs, kap));</code>	Stores wheel torque outputs from <code>my_model.m</code> .
473	<code>MU(:,k) = full(model.f_mu(xs, us, zs, kap));</code>	Stores tyre utilization ratios from <code>my_model.m</code> .
474	<code>LOAD(:,k) = full(model.f_load(xs, us, zs, kap));</code>	Stores load-related outputs from <code>my_model.m</code> .
475	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
476	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
477	<code>sim = struct();</code>	Creates the returned simulation structure.
478	<code>sim.name = name;</code>	Computes or assigns <code>sim.name</code> in <code>postprocess_solution</code> . The exact meaning follows from the variable name and the surrounding function.
479	<code>sim.t = t(:).';</code>	Computes or assigns <code>sim.t</code> in <code>postprocess_solution</code> . The exact meaning follows from the variable name and the surrounding function.
480	<code>sim.x_scaled = X;</code>	Stores scaled states.
481	<code>sim.x_phys = X .* model.x_s(:);</code>	Converts scaled states back to physical units by multiplying by <code>x_s</code> .
482	<code>sim.z_scaled = Z;</code>	Stores scaled loads.
483	<code>sim.z_phys = Z .* model.z_s(:);</code>	Converts scaled loads back to Newtons by multiplying by <code>z_s</code> .
484	<code>sim.u_scaled = U;</code>	Stores scaled controls.
485	<code>sim.u_phys = U .* model.u_s(:);</code>	Converts scaled controls to physical units by multiplying by <code>u_s</code> .
486	<code>sim.h_load = H;</code>	Stores load residual histories.
487	<code>sim.acc = ACC;</code>	Stores acceleration diagnostic histories.
488	<code>sim.torque = TORQUE;</code>	Stores wheel torque histories.
489	<code>sim.mu = MU;</code>	Stores tyre utilization histories.
490	<code>sim.load = LOAD;</code>	Stores load diagnostic histories.
491	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

postprocess_dae_solution (lines 494-531)

Line	Source	Explanation
494	<code>function sim = postprocess_dae_solution(name, t, X, Z, ctrl, veh, model, opts)</code>	Begins postprocessing for the true DAE solution, where <code>Z</code> is already part of the solver output.
495	<code>N = numel(t);</code>	Computes or assigns <code>N</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.

Line	Source	Explanation
496	<code>U = zeros(3, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
497	<code>H = zeros(model.nz, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
498	<code>ACC = zeros(7, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
499	<code>TORQUE = zeros(4, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
500	<code>MU = zeros(4, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
501	<code>LOAD = zeros(11, N);</code>	Preallocates an array of zeros for efficiency and predictable dimensions.
502	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
503	<code>for k = 1:N</code>	Loops through saved DAE output samples.
504	<code> xs = X(:,k);</code>	Computes or assigns xs in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
505	<code> zs = Z(:,k);</code>	Uses DAE-returned algebraic loads directly.
506	<code> kap = eval_kappa(opts, t(k), xs);</code>	Computes or assigns kap in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
507	<code> us = make_applied_control_scaled(t(k), xs, ctrl, veh, model, opts);</code>	Computes or assigns us in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
508	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
509	<code> U(:,k) = us;</code>	Computes or assigns U(:,k) in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
510	<code> H(:,k) = full(model.f_hload(xs, us, zs, kap));</code>	Computes residual to diagnose the DAE solution, not to repair it.
511	<code> ACC(:,k) = full(model.f_acc(xs, us, zs, kap));</code>	Evaluates a CasADi function and converts the result to a full MATLAB numeric array.
512	<code> TORQUE(:,k) = full(model.f_torque(xs, us, zs, kap));</code>	Evaluates a CasADi function and converts the result to a full MATLAB numeric array.
513	<code> MU(:,k) = full(model.f_mu(xs, us, zs, kap));</code>	Evaluates a CasADi function and converts the result to a full MATLAB numeric array.
514	<code> LOAD(:,k) = full(model.f_load(xs, us, zs, kap));</code>	Evaluates a CasADi function and converts the result to a full MATLAB numeric array.
515	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
516	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
517	<code>sim = struct();</code>	Creates the returned DAE simulation structure.
518	<code>sim.name = name;</code>	Computes or assigns sim.name in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
519	<code>sim.t = t(:).';</code>	Computes or assigns sim.t in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
520	<code>sim.x_scaled = X;</code>	Computes or assigns sim.x_scaled in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
521	<code>sim.x_phys = X .* model.x_s(:);</code>	Converts DAE scaled states to physical units.
522	<code>sim.z_scaled = Z;</code>	Computes or assigns sim.z_scaled in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
523	<code>sim.z_phys = Z .* model.z_s(:);</code>	Converts DAE scaled loads to Newtons.

Line	Source	Explanation
524	<code>sim.u_scaled = U;</code>	Computes or assigns <code>sim.u_scaled</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
525	<code>sim.u_phys = U .* model.u_s(:);</code>	Computes or assigns <code>sim.u_phys</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
526	<code>sim.h_load = H;</code>	Computes or assigns <code>sim.h_load</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
527	<code>sim.acc = ACC;</code>	Computes or assigns <code>sim.acc</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
528	<code>sim.torque = TORQUE;</code>	Computes or assigns <code>sim.torque</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
529	<code>sim.mu = MU;</code>	Computes or assigns <code>sim.mu</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
530	<code>sim.load = LOAD;</code>	Computes or assigns <code>sim.load</code> in <code>postprocess_dae_solution</code> . The exact meaning follows from the variable name and the surrounding function.
531	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

plot_simple_results (lines 534-617)

Line	Source	Explanation
534	<code>function plot_simple_results(sim_ode15s, sim_fixed, sim_ode15i_dae, ctrl, veh)</code>	Begins plotting helper.
535	<code>has_dae = ~isempty(sim_ode15i_dae);</code>	Checks whether the DAE result exists, so plots and legends can include or omit it.
536	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
537	<code>figure('Name','Simple time simulation: speed comparison');</code>	Creates the speed comparison figure.
538	<code>plot(sim_ode15s.t, 3.6*sim_ode15s.x_phys(1,:), 'LineWidth', 1.2); hold on; grid on;</code>	Plots ode15s longitudinal speed in km/h.
539	<code>plot(sim_fixed.t, 3.6*sim_fixed.x_phys(1,:), '--', 'LineWidth', 1.0);</code>	Plots RK4 longitudinal speed in km/h with a dashed style.
540	<code>if has_dae</code>	Starts a conditional branch in <code>plot_simple_results</code> .
541	<code> plot(sim_ode15i_dae.t, 3.6*sim_ode15i_dae.x_phys(1,:), ':', 'LineWidth', 1.5);</code>	Plots DAE longitudinal speed if available.
542	<code> legend('ode15s reduced ODE','fixed RK4 reduced ODE','ode15i true DAE');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
543	<code>else</code>	Alternative branch of the preceding if/switch logic.
544	<code> legend('ode15s reduced ODE','fixed RK4 reduced ODE');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
545	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
546	<code>xlabel('t [s]'); ylabel('u_x [km/h]'); title('Longitudinal speed');</code>	Labels the speed figure.
547	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.

Line	Source	Explanation
548	<code>figure('Name','Simple time simulation: selected states comparison');</code>	Creates the selected-state comparison figure.
549	<code> tiledlayout(3,1);</code>	Uses three stacked tiles for speed, yaw rate, and course angle.
550	<code> nexttile;</code>	Controls the tiled subplot layout for grouped plots.
551	<code> plot(sim_ode15s.t, 3.6*sim_ode15s.x_phys(1,:), 'LineWidth', 1.1); hold on; grid on;</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
552	<code> plot(sim_fixed.t, 3.6*sim_fixed.x_phys(1,:), '--');</code>	Adds a plotted curve to the current MATLAB figure.
553	<code> if has_dae, plot(sim_ode15i_dae.t, 3.6*sim_ode15i_dae.x_phys(1,:), ':', 'LineWidth', 1.4); end</code>	Starts a conditional branch in plot_simple_results.
554	<code> xlabel('t [s]'); ylabel('u_x [km/h]'); title('Speed');</code>	Adds axis labels or a plot title.
555	<code> nexttile;</code>	Controls the tiled subplot layout for grouped plots.
556	<code> plot(sim_ode15s.t, rad2deg(sim_ode15s.x_phys(3,:)), 'LineWidth', 1.1); hold on; grid on;</code>	Plots yaw rate in degrees per second.
557	<code> plot(sim_fixed.t, rad2deg(sim_fixed.x_phys(3,:), '--');</code>	Adds a plotted curve to the current MATLAB figure.
558	<code> if has_dae, plot(sim_ode15i_dae.t, rad2deg(sim_ode15i_dae.x_phys(3,:), ':', 'LineWidth', 1.4); end</code>	Starts a conditional branch in plot_simple_results.
559	<code> xlabel('t [s]'); ylabel('r [deg/s]'); title('Yaw rate');</code>	Adds axis labels or a plot title.
560	<code> nexttile;</code>	Controls the tiled subplot layout for grouped plots.
561	<code> plot(sim_ode15s.t, rad2deg(sim_ode15s.x_phys(9,:)), 'LineWidth', 1.1); hold on; grid on;</code>	Plots course angle xi in degrees.
562	<code> plot(sim_fixed.t, rad2deg(sim_fixed.x_phys(9,:), '--');</code>	Adds a plotted curve to the current MATLAB figure.
563	<code> if has_dae, plot(sim_ode15i_dae.t, rad2deg(sim_ode15i_dae.x_phys(9,:), ':', 'LineWidth', 1.4); end</code>	Starts a conditional branch in plot_simple_results.
564	<code> xlabel('t [s]'); ylabel('\xi [deg]'); title('Course angle');</code>	Adds axis labels or a plot title.
565	<code> if has_dae</code>	Starts a conditional branch in plot_simple_results.
566	<code> legend('ode15s reduced ODE','fixed RK4 reduced ODE','ode15i true DAE','Location','best');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
567	<code> else</code>	Alternative branch of the preceding if/switch logic.
568	<code> legend('ode15s reduced ODE','fixed RK4 reduced ODE','Location','best');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
569	<code> end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
570	<code> [blank]</code>	Blank line used to visually separate logical pieces of the code.
571	<code> figure('Name','Simple time simulation: controls');</code>	Creates the control-history figure.
572	<code> plot(ctrl.t, ctrl.Tt_cmd, 'LineWidth', 1.2); hold on; grid on;</code>	Plots drive torque command.
573	<code> yyaxis right; plot(ctrl.t, rad2deg(ctrl.delta_cmd), 'LineWidth', 1.2);</code>	Uses a right y-axis for steering angle in degrees.

Line	Source	Explanation
574	<code>xlabel('t [s]'); yyaxis left; ylabel('T_t command [Nm]'); yyaxis right; ylabel('\delta command [deg]');</code>	Adds axis labels or a plot title.
575	<code>title('Arbitrary command histories');</code>	Adds axis labels or a plot title.
576	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
577	<code>figure('Name','Simple time simulation: tyre loads comparison');</code>	Creates the tyre-load comparison figure.
578	<code>tiledlayout(2,2);</code>	Uses a 2-by-2 tile layout, one tyre per tile.
579	<code>labels = {'FL','FR','RL','RR'};</code>	Defines the tyre labels FL, FR, RL, RR.
580	<code>for i = 1:4</code>	Loops over the four tyres.
581	<code>nexttile;</code>	Controls the tiled subplot layout for grouped plots.
582	<code>plot(sim_ode15s.t, sim_ode15s.z_phys(i,:), 'LineWidth', 1.1); hold on; grid on;</code>	Plots ode15s load for the current tyre.
583	<code>plot(sim_fixed.t, sim_fixed.z_phys(i,:), '--');</code>	Plots RK4 load for the current tyre.
584	<code>if has_dae, plot(sim_ode15i_dae.t, sim_ode15i_dae.z_phys(i,:), ':', 'LineWidth', 1.4); end</code>	Plots DAE load for the current tyre if available.
585	<code>xlabel('t [s]'); ylabel('F_z [N]'); title(labels{i});</code>	Adds axis labels or a plot title.
586	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
587	<code>if has_dae</code>	Starts a conditional branch in plot_simple_results.
588	<code>legend('ode15s reduced ODE','fixed RK4 reduced ODE','ode15i true DAE','Location','best');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
589	<code>else</code>	Alternative branch of the preceding if/switch logic.
590	<code>legend('ode15s reduced ODE','fixed RK4 reduced ODE', 'Location','best');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
591	<code>end</code>	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
592	<code>[blank]</code>	Blank line used to visually separate logical pieces of the code.
593	<code>figure('Name','Simple time simulation: load residual comparison');</code>	Creates the algebraic load residual figure.
594	<code>semilogy(sim_ode15s.t, max(abs(sim_ode15s.h_load),[],1) + eps, 'LineWidth', 1.2); hold on; grid on;</code>	Plots max absolute h_load over tyres for ode15s on a logarithmic scale.
595	<code>semilogy(sim_fixed.t, max(abs(sim_fixed.h_load),[],1) + eps, '--', 'LineWidth', 1.0);</code>	Plots the same residual for RK4.
596	<code>if has_dae</code>	Starts a conditional branch in plot_simple_results.
597	<code>semilogy(sim_ode15i_dae.t, max(abs(sim_ode15i_dae.h_load),[],1) + eps, ':', 'LineWidth', 1.5);</code>	Plots the same residual for the DAE solution if available.
598	<code>legend('ode15s reduced ODE','fixed RK4 reduced ODE','ode15i true DAE');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
599	<code>else</code>	Alternative branch of the preceding if/switch logic.
600	<code>legend('ode15s reduced ODE','fixed RK4 reduced ODE');</code>	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.

Line	Source	Explanation
601	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
602	xlabel('t [s]'); ylabel('max h_{load} [-]'); title('Tyre-load algebraic residual');	Labels the residual plot.
603	[blank]	Blank line used to visually separate logical pieces of the code.
604	if has_dae	Only computes DAE-relative speed error if the DAE result exists.
605	% Compare the two reduced-ODE results to the DAE result at common DAE times.	Comment/documentation: Compare the two reduced-ODE results to the DAE result at common DAE times.
606	tx = sim_ode15i_dae.t;	Uses DAE output times as the common comparison grid.
607	ux_ode = interp1(sim_ode15s.t, sim_ode15s.x_phys(1,:), tx, 'linear', 'extrap');	Interpolates ode15s speed to DAE times.
608	ux_rk4 = interp1(sim_fixed.t, sim_fixed.x_phys(1,:), tx, 'linear', 'extrap');	Interpolates RK4 speed to DAE times.
609	ux_dae = sim_ode15i_dae.x_phys(1,:);	Reads DAE speed at the same times.
610	figure('Name','Simple time simulation: speed difference to DAE');	Creates the speed-difference figure.
611	plot(tx, 3.6*(ux_ode - ux_dae), 'LineWidth', 1.2); hold on; grid on;	Plots ode15s minus DAE speed error.
612	plot(tx, 3.6*(ux_rk4 - ux_dae), '--', 'LineWidth', 1.0);	Plots RK4 minus DAE speed error.
613	xlabel('t [s]'); ylabel('\Delta u_x [km/h]');	Labels speed difference in km/h.
614	legend('ode15s reduced ODE - ode15i DAE','fixed RK4 reduced ODE - ode15i DAE');	Calls or configures MATLAB ode15s, a variable-step stiff ODE solver.
615	title('Speed difference relative to true DAE solution');	Titles the DAE-reference error plot.
616	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.
617	end	Ends the current function, loop, switch, or conditional block, depending on the active nesting level.

Appendix B. Section separators outside function ranges

Line	Source	Explanation
100	[blank]	Blank line used to visually separate logical pieces of the code.
101	%% ----- -----	Comment/documentation: -----
138	[blank]	Blank line used to visually separate logical pieces of the code.
139	%% ----- -----	Comment/documentation: -----
169	[blank]	Blank line used to visually separate logical pieces of the code.
170	%% ----- -----	Comment/documentation: -----

Line	Source	Explanation
195	[blank]	Blank line used to visually separate logical pieces of the code.
196	%% ----- -----	Comment/documentation: -----
216	[blank]	Blank line used to visually separate logical pieces of the code.
217	%% ----- -----	Comment/documentation: -----
225	[blank]	Blank line used to visually separate logical pieces of the code.
226	%% ----- -----	Comment/documentation: -----
235	[blank]	Blank line used to visually separate logical pieces of the code.
236	%% ----- -----	Comment/documentation: -----
254	[blank]	Blank line used to visually separate logical pieces of the code.
255	%% ----- -----	Comment/documentation: -----
265	[blank]	Blank line used to visually separate logical pieces of the code.
266	%% ----- -----	Comment/documentation: -----
337	[blank]	Blank line used to visually separate logical pieces of the code.
338	%% ----- -----	Comment/documentation: -----
367	[blank]	Blank line used to visually separate logical pieces of the code.
368	%% ----- -----	Comment/documentation: -----
431	[blank]	Blank line used to visually separate logical pieces of the code.
432	%% ----- -----	Comment/documentation: -----
448	[blank]	Blank line used to visually separate logical pieces of the code.
449	%% ----- -----	Comment/documentation: -----
492	[blank]	Blank line used to visually separate logical pieces of the code.
493	%% ----- -----	Comment/documentation: -----
532	[blank]	Blank line used to visually separate logical pieces of the code.
533	%% ----- -----	Comment/documentation: -----